
CIS437 - Cloud Computing

– Deploying Cloud Applications from Cloud Shell –

— Instructor: Yong Zhuang —

yong.zhuang@gvsu.edu

Last Class

- What is Cloud Shell?
- Basic Linux commands
- `gcloud` CLI
- Manage a Compute Engine VM from the command line
- Connect to a VM with SSH
- Cloud Storage from the command line

Agenda

- Recovering from a broken SSH configuration
- Serial console
- What is App Engine?
- Deploy a Python/Flask web app
- Test and redeploy an application
- Cloud Storage permissions and public access
- Discussion / attendance quiz

Example 4: Recovering from a Broken SSH Configuration

One of the most common problems you'll have with VMs (i.e., Compute Engine)

How does this happen?

1. You create a VM
2. You connect to the VM and spend a lot of time getting it just so
3. You enable the firewall
4. You close the SSH session
5. ...
6. You re-open the SSH session?
7. **SSH connection fails**
8. You re-open the SSH session?
9. ...

Normal SSH connection no longer works !!!



How can we recover access?

How Do We Lock Ourselves Out?

You create a VM



You connect to the VM and configure it



You enable the firewall *But forget to allow SSH (TCP 22)*



You close the current SSH session



...



You try to reconnect



SSH connection fails

Serial console recovery steps

Shut the machine off

This is assuming you made an Ubuntu VM using UFW as the host firewall. - you'd have to use whichever firewall you enabled for other server OSs)



Edit it and click the checkbox for remote access via the serial console



Configure temporary local login credentials using the recovery method in the **Official Doc**, replacing USERNAME with your Google Cloud username and password with a temporary one



Save it, turn on your machine, connect via serial console, login with the username/pw you just added



```
sudo ufw allow 22/tcp
```



Shut off, edit machine again, disable serial and remove bash script, done

[Cloud console](#)

Official Doc: [Troubleshooting using the serial console](#)

What just happened?

- Normal SSH path was unavailable
- Serial console provided alternate access
- We corrected the VM's firewall/configuration
- Normal SSH access was restored
- Temporary recovery settings should be removed afterward

Normal situation

Laptop / Cloud Shell

↓ SSH :22
VM

SSH broken

Laptop / Cloud Shell

✗
VM

Recovery

Serial Console



VM



Fix SSH/firewall



Normal SSH restored

Example 5: Deploying a Web App with App Engine

- Platform as a Service (PaaS)
- Deploy application code without managing a VM directly
- Google manages much of the runtime infrastructure
- We will use a simple Python/Flask app

Welcome to App Engine

Build scalable apps in any language on Google's infrastructure

Deploying an App Engine app

- App Engine is initialized within a Google Cloud project
- The App Engine application can be **disabled and re-enabled**
- Deployed versions/services can be managed separately
- Deleting the entire project removes all associated resources
- For classroom demos, a dedicated project makes cleanup easier

Use a Dedicated Project

For a temporary classroom demo, a dedicated project makes cleanup easier.

New Project

Project name *
cloudshell-appengine-test

Project ID *
unique-machine-508912-d6

Project ID can have lowercase letters, digits, or hyphens. It must start with a lowercase letter and end with a letter or number.

Organization *
mail.gvsu.edu

Select an organization to attach it to a project. This selection can't be changed later.

Parent resource *
mail.gvsu.edu

Select the organization or folder where this resource will be created

Create Cancel

But we still have to deal with this!



```
CLOUD SHELL
Terminal (fluted-current-507304-t3) x + v
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to fluted-current-507304-t3.
Use `gcloud config set project [PROJECT_ID]` to change to a different project.
zhuangyo@cloudshell:~ (fluted-current-507304-t3) $
```



How?

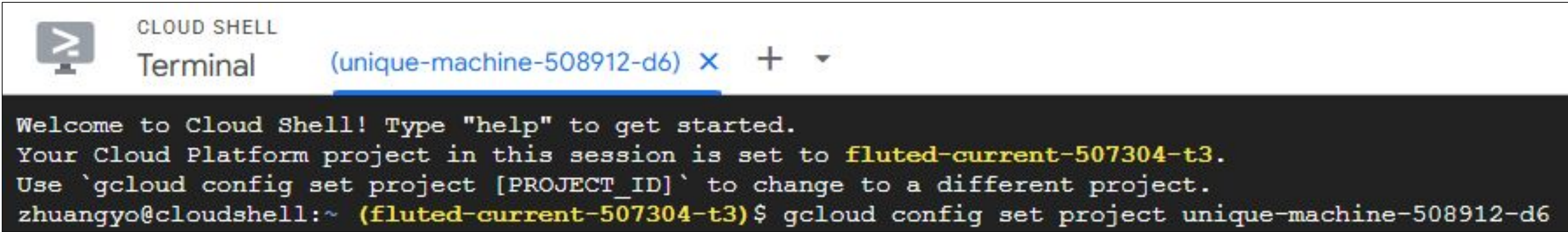
Cloud console

Use a Dedicated Project

Switch the project

Recent Starred All				
Name		Type	ID	
✓	 cloudshell-appengine-test ⓘ	Project	unique-machine-508912-d6	☆
	 zhuang-CloudAppDev ⓘ	Project	fluted-current-507304-t3	☆

Or



The screenshot shows a Cloud Shell terminal window. The title bar includes a terminal icon, the text 'CLOUD SHELL', and a tab labeled 'Terminal (unique-machine-508912-d6)'. The terminal content shows a welcome message and instructions. The current project is 'fluted-current-507304-t3'. The user has entered the command 'gcloud config set project unique-machine-508912-d6' to switch to the 'unique-machine-508912-d6' project.

```
Cloud Shell
Terminal (unique-machine-508912-d6) × + ▾

Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to fluted-current-507304-t3.
Use `gcloud config set project [PROJECT_ID]` to change to a different project.
zhuangyo@cloudshell:~ (fluted-current-507304-t3)$ gcloud config set project unique-machine-508912-d6
```

`gcloud config set project PROJECT_ID`

then,

`gcloud config get-value project`

[Cloud console](#)

Codelab

We'll use Steps 1–7 of the Codelab below as our deployment walkthrough.

(After we get to the code, move on to the next slide because we have **EXTRA CONTENT TO SEE**)

code → dependencies → local test → deploy → browser.

[Cloud console](#)

codelab: [Getting started with App Engine \(Python 3\)](#)

Testing!

Prior to deploying to App Engine, we can test it locally

```
zhuangyo@cloudshell:~/helloworld (unique-machine-508912-d6)$ python3 -m venv appengine-demo
zhuangyo@cloudshell:~/helloworld (unique-machine-508912-d6)$ source appengine-demo/bin/activate
(appengine-demo) zhuangyo@cloudshell:~/helloworld (unique-machine-508912-d6)$
pip install -r requirements.txt
```

First, install the Python libraries(do this in a virtual environment...)

- `python3 -m venv VENV-NAME`
- `source VENV-NAME/bin/activate`
- `pip install -r requirements.txt`

And then run the program

- `python3 main.py`

And then open the proxied debug view

[Cloud console](#)

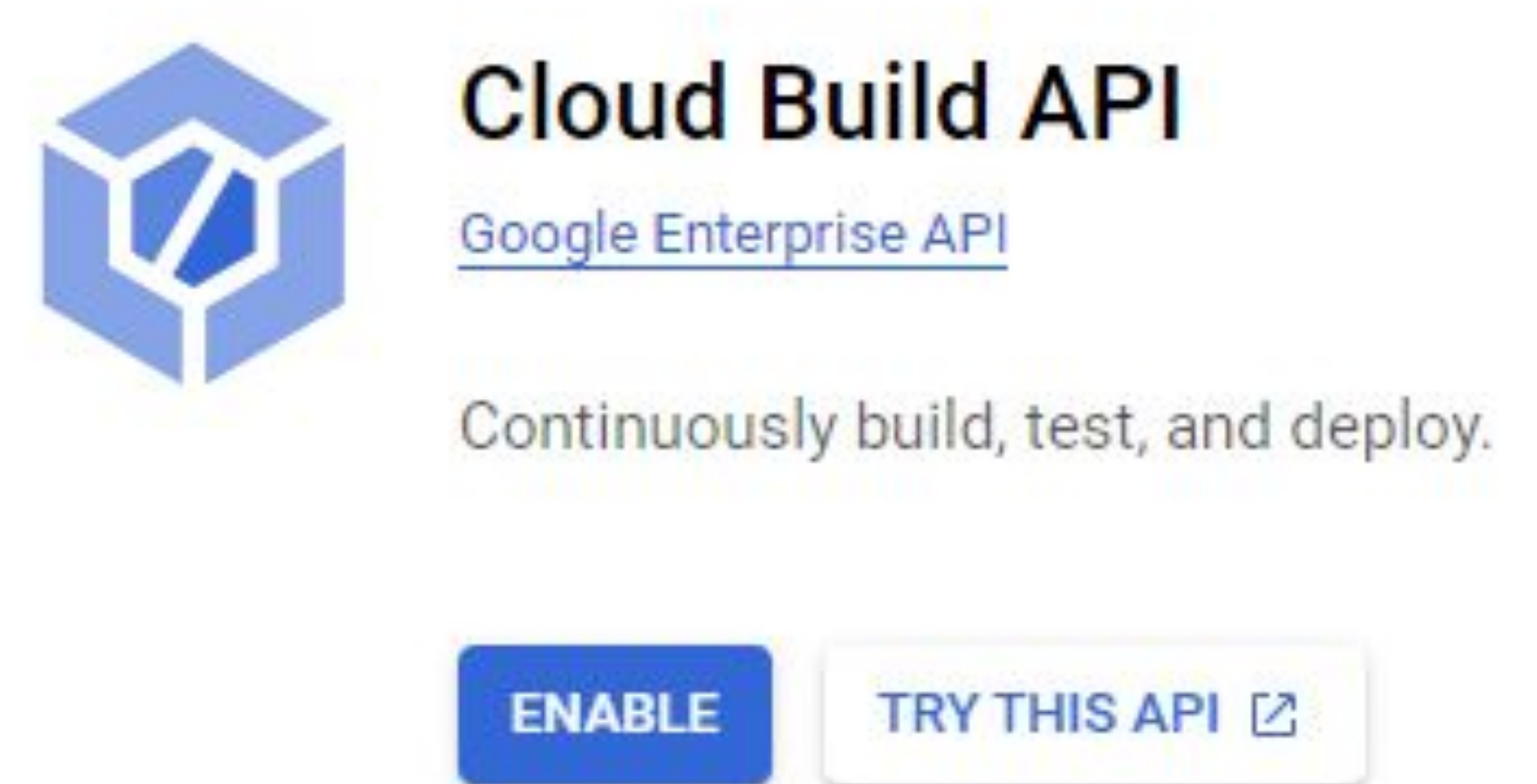


If Deployment Fails

If we see this:

ERROR: (gcloud.app.deploy) Permissions error fetching application [.....]. Please make sure that you have permission to view applications on the project and that <email> has the App Engine Deployer (roles/appengine.deployer) role

Then go to Cloud Build and enable it



IAM troubleshooting

If we see this:

ERROR: (gcloud.app.deploy) Permissions error fetching application [.....]. Please make sure that you have permission to view applications on the project and that <email> has the App Engine Deployer (roles/appengine.deployer) role

Then go to IAM and give yourself the App Engine Deployer role

Grant the minimum role required by the error. Do not automatically grant Admin.

Edit access to "cloudshell-appengine-test"

Principal ?

zhuangyo@mail.gvsu.edu

Project

cloudshell-appengine-test

Assign roles

Roles are composed of sets of permissions and determine what the principal can do with this resource. [Learn more](#)

Role

Owner

IAM condition (optional) ?

+ Add IAM condition

Full access to most Google Cloud resources. See the list of included permissions.

Select a role

IAM condition (optional) ?

Search App engine

Top results

App Engine Admin

Full management of App Engine apps (but not storage).

Appengine Editor

Editor role for appengine

App Engine Code Viewer

Ability to view App Engine app status and deployed source code.

App Engine Viewer

Ability to view App Engine app status

[Manage Roles](#)

End result



Hello World!

So what do we have here?

- A Flask application running on the cloud
 - Could become anything! But, you now have a backend server that can do all kinds of incredible things
- Anything supported by App Engine can be hosted here!
 - Per usual though, check the \$\$ guide

When finished, disable the application or delete the dedicated demo project, and monitor usage/cost.

Updating!

We have a neat thing in a bucket somewhere...

Update the HTML page:

- Grab the URL to the GIF and paste into the HTML

```

```

Redeploy app

```
$ gcloud app deploy --quiet
```

Cloud Storage URI:
gs://BUCKET_NAME/IMG_NAME.gif

Browser URL:
https://storage.googleapis.com/BUCKET_NAME/IMG_NAME.gif

Official Doc: [Access to public objects](#)

Generate / upload GIF if you do not have

Download & Rename gif to Google Shell

wget

<https://gifdb.com/images/high/obi-wan-kenobi-well-hello-there-hzgui7yr5ketac2c.gif>

```
mv obi-wan-kenobi-well-hello-there-hzgui7yr5ketac2c.gif hello.gif
```

Create bucket an upload GIF

```
gcloud storage buckets create gs://zhuang-demo-20260915 --location=us-east1
```

```
gcloud storage cp hello.gif gs://zhuang-demo-20260915
```

```
gcloud storage ls gs://zhuang-demo-20260915
```



But wait...

Didn't work?

What's wrong?



Permissions!

Need to be **very careful with this**

- Public access means anybody/thing can access
- What happens if somebody bots your site?
 - \$\$\$

For this controlled demo, we will intentionally make this object public

- Alternatively, you can grant fine-grained access to whatever cloud product needs the file

```
$ gcloud storage objects update gs://BUCKET_NAME/OBJECT_NAME  
--add-acl-grant=entity=AllUsers,role=READER
```

Update URL and redeploy!

```
def hello():  
    """Return a friendly HTTP  
    greeting."""  
    return "Hello World!<br> <img  
    src='https://storage.googleapis.com/B  
    UCKET_NAME/OBJECT_NAME' />"
```

Redeploy app: `$ gcloud app deploy --quiet`

Official Doc: [Create and manage access control lists \(ACLs\)](#) or [Make data public](#)